



**UNIVERSIDAD DE
GUADALAJARA**
Red Universitaria e Institución Benemérita de Jalisco



División de Tecnologías para la Integración Ciber-Humana

Entrega Avance - Pipeline Distribuido IoT/Edge con Tolerancia a Fallos

Alumnos

Casillas Rendón Jennyfher Jacqueline

Huerta Murillo Alejandro Saul

Lona Avalos Cristian Josué

Programación de Sistemas Avanzados

D01 2026-A

Profesor

López Arce Delgado Jorge Ernesto

Guadalajara, Jalisco, 07 de mayo de 2026

Índice

Índice.....	2
Introducción.....	3
Objetivo.....	3
Objetivo General.....	3
Objetivos Particulares.....	3
Desarrollo.....	4
Topología de red.....	4
Diagrama de la topología.....	4
Descripción del rol de cada nodo.....	4
Evidencia de tc netem.....	6
Antes.....	6
Después.....	7
Declaración de nivel de red y justificación.....	8
mTLS entre contenedores en Rust: certificados autofirmados, verificación mutua entre coordinator y edge nodes.....	9
Kubernetes k3s.yml.....	10
docker-compose.yml con los tres tipos de contenedor definidos.....	13
Reglas tc netem.....	14
Validar cada escenario con iperf3 antes de ejecutar el sistema distribuido. Incluir capturas de salida.....	15
Salida Normal.....	15
Comandos exactos utilizados para activar y desactivar cada escenario.....	16
baseline.sh.....	16
enlace_limitado.sh.....	16
falla_nodo.sh.....	19
latencia_iot.sh.....	21
perdida_paquetes.sh.....	23
status.sh.....	23
Implementación base en Rust.....	24
Cargo.toml.....	24
Estructuras de mensaje con serde.....	25
Prototipo.....	26
Plan de ejecución.....	30
Pruebas de tolerancia a fallos.....	30
Medición de métricas de rendimiento.....	31
Riesgos técnicos identificados.....	31
Roles.....	32
Cronograma.....	32
Product Backlog (Historias de Usuario).....	32

Sprint Backlog.....	33
Minutas de Reunión.....	33
Retos encontrados hasta el momento y cómo se resolvieron.....	41
Conclusión.....	42
Referencias.....	42

Introducción

El presente proyecto tiene como finalidad el diseño e implementación de un sistema distribuido basado en un pipeline IoT/Edge con tolerancia a fallos, utilizando Rust, Docker y Kubernetes. Este sistema simula un entorno real donde múltiples sensores generan datos de forma continua, los cuales son procesados por nodos intermedios, que en este caso son los edge y posteriormente agregados por un coordinador central.

El proyecto se enfoca en un modelo basado en eventos en tiempo real, lo que introduce retos importantes como la concurrencia, la sincronización entre nodos y la resiliencia ante fallas. Además, se incorpora la simulación de degradación de red mediante con tc netem, permitiendo evaluar el comportamiento del sistema bajo escenarios adversos.

Objetivo

Objetivo General

Diseñar, implementar y evaluar un pipeline distribuido IoT/Edge tolerante a fallos, capaz de procesar datos en tiempo real sobre una infraestructura basada en contenedores y red VPN, demostrando su funcionamiento bajo diferentes condiciones de red degradada y garantizando la recuperación ante fallas de los nodos.

Objetivos Particulares

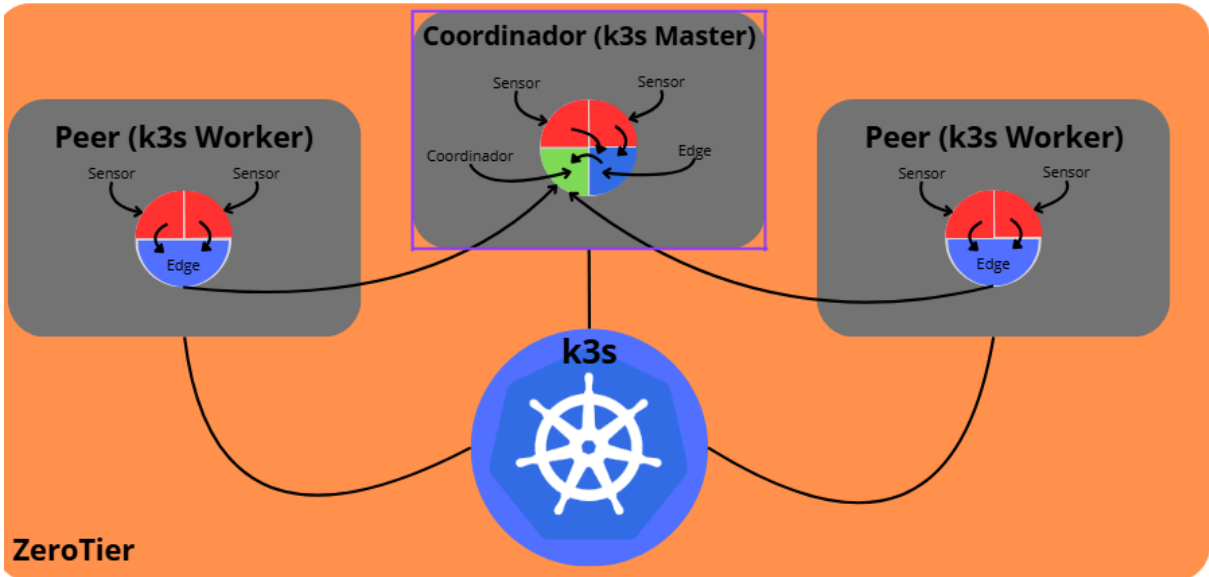
- Implementar un sistema distribuido en Rust compuesto por sensores, nodos edge y un coordinador, que permita el flujo continuo de datos en tiempo real.
- Configurar una infraestructura basada en Docker en una red distribuida que permita la comunicación entre los distintos componentes del sistema.
- Simular condiciones de red adversas mediante tc netem (latencia, pérdida de paquetes) para analizar el comportamiento del sistema.
- Implementar mecanismos de tolerancia a fallos, incluyendo detección de nodos caídos y reconexión automática de los mismos.
- Medir y analizar métricas de rendimiento como latencia, throughput y tasa de anomalías en distintos escenarios de red.

Desarrollo

Topología de red

Nuestra arquitectura sigue un patrón Coordinador-Worker implementado sobre una red en ZeroTier y desplegado utilizando Kubernetes para la orquestación de contenedores de docker. La topología es un modelo hub-and-spoke donde el nodo coordinador actúa como punto central de la red, mientras que los nodos worker se conectan a este para el intercambio de información.

Diagrama de la topología



Descripción del rol de cada nodo.

Nodo	Nombre del host	Rol
Nodo 1	coordinador	Ejecuta el coordinador encargado de recibir y agregar los datos procesados por los nodos edge, calcular métricas y detectar fallas mediante heartbeats. Además, en este nodo también se despliegan 2 sensores y 1 nodo edge como parte del entorno distribuido gestionado con Kubernetes.

Nodo 2	peer-1	Nodo worker que ejecuta 2 sensores encargados de generar datos sintéticos y un nodo edge que procesa dichos datos (promedio móvil, detección de anomalías) antes de enviarlos al coordinador. Este nodo forma parte del clúster de Kubernetes, permitiendo escalabilidad y recuperación automática.
Nodo 3	peer-2	Nodo worker con la misma configuración que Peer 1. Ejecuta 2 sensores y 1 nodo edge, contribuyendo al procesamiento distribuido del sistema. También está integrado al clúster de Kubernetes, asegurando tolerancia a fallos y balanceo de carga.

Sensor: genera datos sintéticos periódicamente y los envían a los nodos edge.

Edge: recibe datos de múltiples sensores, realizan procesamiento local y reenvían la información al coordinador.

Coordinador: centraliza el procesamiento global, agrega métricas, detecta fallas en los nodos y expone información del estado del sistema.

Kubernetes permite manejar múltiples réplicas de los nodos, lo que mejora la tolerancia a fallos y la escalabilidad del sistema. En caso de falla de un nodo, Kubernetes puede reiniciarlo automáticamente mientras que el coordinador detecta la desconexión y posterior reconexión, garantizando la continuidad del pipeline.

Evidencia de tc netem

Antes

```
chino@chino ssh ~/Pipeline_Distribuido git:(feature/k3s) 8 files changed, 75 insertions(+), 24 deletions(-)
iperf3 -s

-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.223.113.47, port 60324
[ 5] local 10.223.113.235 port 5201 connected to 10.223.113.47 port 60338
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-1.00      sec   25.6 MBytes  215 Mbits/sec
[ 5]  1.00-2.00      sec   25.8 MBytes  216 Mbits/sec
[ 5]  2.00-3.00      sec   31.8 MBytes  266 Mbits/sec
[ 5]  3.00-4.00      sec   33.9 MBytes  284 Mbits/sec
[ 5]  4.00-5.00      sec   25.5 MBytes  214 Mbits/sec
[ 5]  5.00-6.00      sec   36.0 MBytes  302 Mbits/sec
[ 5]  6.00-7.00      sec   15.4 MBytes  129 Mbits/sec
[ 5]  7.00-8.00      sec   15.2 MBytes  128 Mbits/sec
[ 5]  8.00-9.00      sec   15.2 MBytes  128 Mbits/sec
[ 5]  9.00-10.00     sec   14.9 MBytes  125 Mbits/sec
[ 5] 10.00-10.06     sec    1.00 MBytes  136 Mbits/sec
-----
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-10.06     sec   240 MBytes  200 Mbits/sec                      receiver
-----
Server listening on 5201 (test #2)
-----
```

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 75 insertions(+), 24 deletions(-) (10.174s)
iperf3 -c 10.223.113.235

Connecting to host 10.223.113.235, port 5201
[ 5] local 10.223.113.47 port 60338 connected to 10.223.113.235 port 5201
[ ID] Interval           Transfer     Bitrate      Retr  Cwnd
[ 5]  0.00-1.00      sec   27.9 MBytes  234 Mbits/sec    0   1.27 MBytes
[ 5]  1.00-2.00      sec   24.2 MBytes  203 Mbits/sec    0   2.49 MBytes
[ 5]  2.00-3.00      sec   34.6 MBytes  290 Mbits/sec   335   2.62 MBytes
[ 5]  3.00-4.00      sec   33.1 MBytes  278 Mbits/sec   11   1.96 MBytes
[ 5]  4.00-5.00      sec   25.0 MBytes  210 Mbits/sec    0   2.07 MBytes
[ 5]  5.00-6.00      sec   37.0 MBytes  310 Mbits/sec    0   2.15 MBytes
[ 5]  6.00-7.00      sec   14.6 MBytes  123 Mbits/sec   157   762 KBytes
[ 5]  7.00-8.00      sec   15.9 MBytes  133 Mbits/sec    0   837 KBytes
[ 5]  8.00-9.00      sec   14.6 MBytes  123 Mbits/sec    0   888 KBytes
[ 5]  9.00-10.00     sec   16.0 MBytes  134 Mbits/sec    0   912 KBytes
-----
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.00     sec   243 MBytes  204 Mbits/sec   503
[ 5]  0.00-10.06     sec   240 MBytes  200 Mbits/sec
-----
sender
receiver

iperf Done.
```

Después

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 75 insertions(+), 24 deletions(-) (0.058s)
./enlace_limitado.sh ztktivcv7u
Configurando enlace limitado en la interfaz: ztktivcv7u
Escenario activado: Ancho de banda 512kbit + Delay 50ms.
```

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 75 insertions(+), 24 deletions(-) (18.954s)
iperf3 -c 10.223.113.235
Connecting to host 10.223.113.235, port 5201
[ 5] local 10.223.113.47 port 44308 connected to 10.223.113.235 port 5201
[ ID] Interval           Transfer     Bitrate      Retr  Cwnd
[ 5]  0.00-1.00    sec    256 KBytes  2.09 Mbits/sec    0   77.8 KBytes
[ 5]  1.00-2.00    sec     0.00 Bytes  0.00 bits/sec    0   80.5 KBytes
[ 5]  2.00-3.00    sec     0.00 Bytes  0.00 bits/sec    0   83.2 KBytes
[ 5]  3.00-4.00    sec    128 KBytes  1.05 Mbits/sec    0   91.2 KBytes
[ 5]  4.00-5.00    sec     0.00 Bytes  0.00 bits/sec    0   113 KBytes
[ 5]  5.00-6.00    sec    256 KBytes  2.10 Mbits/sec    0   145 KBytes
[ 5]  6.00-7.00    sec    128 KBytes  1.05 Mbits/sec    0   174 KBytes
[ 5]  7.00-8.00    sec     0.00 Bytes  0.00 bits/sec    0   204 KBytes
[ 5]  8.00-9.00    sec    256 KBytes  2.10 Mbits/sec    0   233 KBytes
[ 5]  9.00-10.00   sec     0.00 Bytes  0.00 bits/sec    0   266 KBytes
-----
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]  0.00-10.00   sec    1.00 MBytes  839 Kbits/sec    0
[ 5]  0.00-14.30   sec     768 KBytes  440 Kbits/sec
sender
receiver

iperf Done.
```

```
-----
Server listening on 5201 (test #2)
-----
```

```
Accepted connection from 10.223.113.235, port 44306
```

```
[ 5] local 10.223.113.235 port 5201 connected to 10.223.113.47 port 44308
```

```
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-1.00    sec     0.00 Bytes  0.00 bits/sec
[ 5]  1.00-2.00    sec     0.00 Bytes  0.00 bits/sec
[ 5]  2.00-3.00    sec    128 KBytes  1.05 Mbits/sec
[ 5]  3.00-4.00    sec     0.00 Bytes  0.00 bits/sec
[ 5]  4.00-5.00    sec    128 KBytes  1.05 Mbits/sec
[ 5]  5.00-6.00    sec     0.00 Bytes  0.00 bits/sec
[ 5]  6.00-7.00    sec    128 KBytes  1.05 Mbits/sec
[ 5]  7.00-8.00    sec     0.00 Bytes  0.00 bits/sec
[ 5]  8.00-9.00    sec    128 KBytes  1.05 Mbits/sec
[ 5]  9.00-10.00   sec     0.00 Bytes  0.00 bits/sec
[ 5] 10.00-11.00   sec    128 KBytes  1.05 Mbits/sec
[ 5] 11.00-12.00   sec     0.00 Bytes  0.00 bits/sec
[ 5] 12.00-13.00   sec    128 KBytes  1.05 Mbits/sec
[ 5] 13.00-14.00   sec     0.00 Bytes  0.00 bits/sec
[ 5] 14.00-14.30   sec     0.00 Bytes  0.00 bits/sec
```

```
-----
[ ID] Interval           Transfer     Bitrate
[ 5]  0.00-14.30   sec     768 KBytes  440 Kbits/sec
```

```
receiver
```

```
-----
Server listening on 5201 (test #3)
-----
```

Declaración de nivel de red y justificación.

La infraestructura de red corresponde al Nivel 3, de acuerdo con la clasificación establecida en los lineamientos del proyecto, debido a que todos de los integrantes operamos bajo condiciones de CGNAT

El uso de CGNAT implica que no contamos con una dirección IP pública accesible directamente desde el exterior, lo cual imposibilita establecer conexiones entrantes de manera tradicional, como lo requiere una configuración estándar de VPN tipo hub-and-spoke en ZeroTier. En este contexto, los nodos no pueden exponerse directamente a internet ni aceptar conexiones entrantes sin mecanismos adicionales, lo que limita significativamente la implementación de una red distribuida convencional.

Se consideró la posibilidad de solicitar a los proveedores de internet la asignación de una IP pública o la apertura de puertos específicos, sin embargo, este proceso presenta múltiples limitaciones prácticas. En primer lugar, los proveedores suelen no responder de manera eficiente o oportuna a este tipo de solicitudes cuando provienen de usuarios residenciales. En segundo lugar, este tipo de configuraciones está generalmente orientado a clientes empresariales, implicando costos adicionales o planes de servicio distintos a los disponibles para usuarios domésticos. Finalmente, la apertura manual de puertos representa un riesgo de seguridad considerable ya que no contamos con la experiencia necesaria para gestionar y configurar correctamente la exposición de servicios a internet de esta forma.

Adicionalmente, desde el punto de vista de seguridad, abrir puertos puede incrementar la superficie de ataque, exponiendo los dispositivos a posibles accesos no autorizados. Dado que el entorno del proyecto no está diseñado para operar como infraestructura de producción con medidas de seguridad avanzadas, el equipo consideró que esta alternativa no era adecuada ni segura para su implementación.

Debido a estas restricciones, optamos por utilizar ZeroTier como solución de red virtual, la cual permite la creación de una red privada definida por software sin requerir configuraciones complejas de red ni apertura de puertos. ZeroTier facilita la conectividad entre nodos distribuidos incluso en presencia de CGNAT, estableciendo túneles seguros entre los participantes y permitiendo simular una red local privada sobre internet.

Esta decisión permite cumplir con los objetivos del proyecto en términos de conectividad distribuida, manteniendo al mismo tiempo un enfoque práctico y seguro acorde a las limitaciones reales del entorno de los integrantes. Asimismo, el uso de esta tecnología se complementa con la implementación de Kubernetes para la orquestación de contenedores y con herramientas como tc netem para la simulación de condiciones de red adversas, garantizando que el sistema pueda ser evaluado bajo escenarios representativos.

mTLS entre contenedores en Rust: certificados autofirmados, verificación mutua entre coordinator y edge nodes.

Esta parte quedaría pendiente de implementación y estaría listo para la siguiente entrega.

Kubernetes k3s.yml

```
1  ---
2  # =====
3  # 1. COORDINADOR (Solo en el Master)
4  # =====
5  apiVersion: apps/v1
6  kind: Deployment
7  metadata:
8    name: coordinador-deployment
9  spec:
10    replicas: 1
11    selector:
12      matchLabels:
13        app: coordinador
14    template:
15      metadata:
16        labels:
17          app: coordinador
18      spec:
19        nodeSelector:
20          rol: master
21        tolerations:
22          - key: "node-role.kubernetes.io/master"
23            operator: "Exists"
24            effect: "NoSchedule"
25          - key: "node-role.kubernetes.io/control-plane"
26            operator: "Exists"
27            effect: "NoSchedule"
28        containers:
29          - name: coordinador
30            image: app-coordinador:v1
31            imagePullPolicy: Never
32            ports:
33              - containerPort: 3001
34            env:
35              - name: RUST_LOG
36                value: "info"
37              - name: BIND_ADDR
38                value: "0.0.0.0:3001"
39              - name: TLS_CERT
40                value: "/certs/tls.crt"
41              - name: TLS_KEY
42                value: "/certs/tls.key"
43              - name: TLS_CA
44                value: "/certs/ca.crt"
45            volumeMounts:
46              - name: tls-certs
47                mountPath: /certs
48                readOnly: true
49        volumes:
50          - name: tls-certs
51            secret:
52              secretName: coord-tls
53  ---
54  apiVersion: v1
55  kind: Service
56  metadata:
57    name: coordinador-service
58  spec:
59    selector:
60      app: coordinador
61    ports:
62      - protocol: TCP
63        port: 3001
64        targetPort: 3001
65  ---
66  ---
```

```

1  —
2  # =====
3  # 2. EDGE (DaemonSet - En Master y Spokes)
4  # =====
5  apiVersion: apps/v1
6  kind: DaemonSet
7  metadata:
8    name: edge-daemonset
9  spec:
10   selector:
11     matchLabels:
12       app: edge
13   template:
14     metadata:
15       labels:
16         app: edge
17     spec:
18       nodeSelector:
19         vincular-edge: "si" # Corre en cualquier nodo con esta etiqueta
20       tolerations:
21         - key: "node-role.kubernetes.io/master"
22           operator: "Exists"
23           effect: "NoSchedule"
24         - key: "node-role.kubernetes.io/control-plane"
25           operator: "Exists"
26           effect: "NoSchedule"
27       containers:
28         - name: edge
29           image: app-edge:v1
30           imagePullPolicy: Never
31           ports:
32             - containerPort: 3000
33           env:
34             - name: RUST_LOG
35               value: "info"
36             - name: BIND_ADDR
37               value: "0.0.0.0:3000"
38             - name: COORD_URL
39               value: "https://coordinador-service:3001"
40             - name: TLS_CERT
41               value: "/certs/tls.crt"
42             - name: TLS_KEY
43               value: "/certs/tls.key"
44             - name: TLS_CA
45               value: "/certs/ca.crt"
46           volumeMounts:
47             - name: tls-certs
48               mountPath: /certs
49               readOnly: true
50           volumes:
51             - name: tls-certs
52               secret:
53                 secretName: edge-tls
54   —
55 apiVersion: v1
56 kind: Service
57 metadata:
58   name: edge-service
59 spec:
60   internalTrafficPolicy: Local
61   selector:
62     app: edge
63   ports:
64     - protocol: TCP
65       port: 3000
66       targetPort: 3000
67   —
68   —

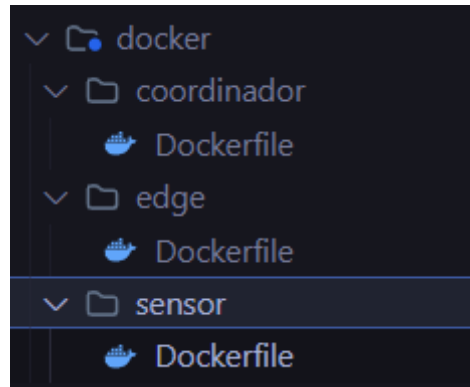
```

```

1 # =====
2 # 3. SENSORES (DaemonSets - S1 y S2 en cada nodo)
3 # =====
4 apiVersion: apps/v1
5 kind: DaemonSet
6 metadata:
7   name: sensor-1-daemonset
8 spec:
9   selector:
10    matchLabels:
11      app: sensor-1
12   template:
13     metadata:
14       labels:
15         app: sensor-1
16     spec:
17       nodeSelector:
18         vincular-edge: "si"
19       tolerations:
20         - key: "node-role.kubernetes.io/master"
21           operator: "Exists"
22           effect: "NoSchedule"
23       containers:
24         - name: sensor
25           image: app-sensor:v1
26           imagePullPolicy: Never
27           env:
28             - name: RUST_LOG
29               value: "info"
30             - name: SENSOR_ID
31               value: "S1"
32             - name: EDGE_URL
33               value: "http://edge-service:3000/api/sensor"
34   ---
35 apiVersion: apps/v1
36 kind: DaemonSet
37 metadata:
38   name: sensor-2-daemonset
39 spec:
40   selector:
41    matchLabels:
42      app: sensor-2
43   template:
44     metadata:
45       labels:
46         app: sensor-2
47     spec:
48       nodeSelector:
49         vincular-edge: "si"
50       tolerations:
51         - key: "node-role.kubernetes.io/master"
52           operator: "Exists"
53           effect: "NoSchedule"
54       containers:
55         - name: sensor
56           image: app-sensor:v1
57           imagePullPolicy: Never
58           env:
59             - name: RUST_LOG
60               value: "info"
61             - name: SENSOR_ID
62               value: "S2"
63             - name: EDGE_URL
64               value: "http://edge-service:3000/api/sensor"

```

docker-compose.yml con los tres tipos de contenedor definidos



```
1  #!/bin/bash
2
3  # Detener el script inmediatamente si algún comando falla
4  set -e
5
6  echo "===== "
7  echo "Iniciando construcción de imágenes... "
8  echo "===== "
9
10 # 1. Construir el Coordinador
11 echo "[1/3] Construyendo app-coordinador:v1..."
12 docker build -t app-coordinador:v1 -f docker/coordinador/Dockerfile .
13
14 # 2. Construir el Edge
15 echo "[2/3] Construyendo app-edge:v1..."
16 docker build -t app-edge:v1 -f docker/edge/Dockerfile .
17
18 # 3. Construir los Sensores
19 echo "[3/3] Construyendo app-sensor:v1..."
20 docker build -t app-sensor:v1 -f docker/sensor/Dockerfile .
21
22 echo "¡Todas las imágenes se construyeron correctamente!"
23 echo "===== "
24
25 # 4. Empaquetar para k3s
26 echo "Empaquetando las imágenes en 'imagenes-rust.tar' para k3s..."
27 docker save app-coordinador:v1 app-edge:v1 app-sensor:v1 -o imagenes-rust.tar
28
29 echo "¡Proceso completado!"
30 echo ""
31 echo "Siguiente paso:"
32 echo "Copia el archivo 'imagenes-rust.tar' a tus nodos (si es necesario) y ejecuta:"
33 echo "  sudo k3s ctr images import imagenes-rust.tar"
```

Reglas tc netem

El clúster k3s corre sobre 3 máquinas físicas distintas, cada una en una red local diferente. Como no existe enlace L2/L3 directo entre ellas, toda la comunicación inter-nodo viaja por una red overlay de ZeroTier: el kubelet, el plano de control, el tráfico de pods (Flannel/CNI sobre la IP del nodo) y los servicios usan exclusivamente la IP que ZeroTier asigna a cada host. Se eligió ZeroTier porque es la VPN que nos permite tener conectividad directa entre las computadoras de los integrantes; sin ella no habría ruta entre los nodos del clúster.

No usamos WireGuard (wg0); ZeroTier es el único transporte entre nodos.

Interfaz sobre la que se aplican las reglas tc netem

Las reglas se aplican sobre la interfaz virtual de ZeroTier de cada host (típicamente `ztXXXXXXXX`, p. ej. `zt7nhx5q7w`).

Justificación: Debido a nuestro problema del CGNAT no tenemos la posibilidad de usar wireguard, además de que zerotier es el encargado de hacer la conexión de nuestra computadoras pudiendo montar el cluster de manera distribuida.

Validar cada escenario con iperf3 antes de ejecutar el sistema distribuido. Incluir capturas de salida.

Salida Normal

```
chino@chino ssh ~
iperf3 -s

-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.223.113.47, port 54568
[ 5] local 10.223.113.235 port 5201 connected to 10.223.113.47 port 54574
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00 sec  37.6 MBytes 315 Mbits/sec
[ 5] 1.00-2.00 sec  40.0 MBytes 335 Mbits/sec
[ 5] 2.00-3.00 sec  35.8 MBytes 300 Mbits/sec
[ 5] 3.00-4.00 sec  37.1 MBytes 311 Mbits/sec
[ 5] 4.00-5.00 sec  16.6 MBytes 140 Mbits/sec
[ 5] 5.00-6.00 sec  15.0 MBytes 126 Mbits/sec
[ 5] 6.00-7.00 sec  15.0 MBytes 126 Mbits/sec
[ 5] 7.00-8.00 sec  15.2 MBytes 128 Mbits/sec
[ 5] 8.00-9.00 sec  15.1 MBytes 127 Mbits/sec
[ 5] 9.00-10.00 sec 15.8 MBytes 132 Mbits/sec
[ 5] 10.00-10.04 sec 512 KBytes 119 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-10.04 sec 244 MBytes 204 Mbits/sec
-----
Server listening on 5201 (test #2)
-----

[ 5] 0.00-10.04 sec 244 MBytes 204 Mbits/sec receiver

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.041s)
clear

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.053s)
./status.sh ztktivcv7u

Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u

qdisc fq_codel 0: root refcnt 2 limit 10240p flows 1024 quantum 1514 target 5ms interval 100ms memory_limit 32Mb ecn drop_batch 64

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (10.132s)
iperf3 -c 10.223.113.235

Connecting to host 10.223.113.235, port 5201
[ 5] local 10.223.113.47 port 54574 connected to 10.223.113.235 port 5201
[ ID] Interval      Transfer    Bitrate  Retr  Cwnd
[ 5] 0.00-1.00 sec  39.6 MBytes 332 Mbits/sec  61   649 KBytes
[ 5] 1.00-2.00 sec  40.2 MBytes 338 Mbits/sec 143   392 KBytes
[ 5] 2.00-3.00 sec  35.6 MBytes 299 Mbits/sec  18   392 KBytes
[ 5] 3.00-4.00 sec  37.4 MBytes 314 Mbits/sec  37   365 KBytes
[ 5] 4.00-5.00 sec  16.2 MBytes 136 Mbits/sec   0   427 KBytes
[ 5] 5.00-6.00 sec  15.4 MBytes 129 Mbits/sec  15   341 KBytes
[ 5] 6.00-7.00 sec  14.5 MBytes 122 Mbits/sec   0   403 KBytes
[ 5] 7.00-8.00 sec  15.4 MBytes 129 Mbits/sec   0   448 KBytes
[ 5] 8.00-9.00 sec  15.4 MBytes 129 Mbits/sec   0   494 KBytes
[ 5] 9.00-10.00 sec  15.4 MBytes 129 Mbits/sec   0   539 KBytes
-----
[ ID] Interval      Transfer    Bitrate  Retr
[ 5] 0.00-10.00 sec 245 MBytes 206 Mbits/sec 274
[ 5] 0.00-10.04 sec 244 MBytes 204 Mbits/sec
-----
sender
receiver

iperf Done
```

Comandos exactos utilizados para activar y desactivar cada escenario

baseline.sh

Restaura la red limpia eliminando cualquier qdisc activo. Es el comando de desactivación universal para todos los escenarios netem.

```
# Activar (restaurar baseline)
sudo ./netem/baseline.sh <ZT_IFACE>
```

```
# Comando equivalente:
sudo tc qdisc del dev <ZT_IFACE> root
```

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.052s)
./baseline.sh ztktivcv7u
Limpiando reglas de red en la interfaz: ztktivcv7u
Red restaurada a su estado base (sin degradación).

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.053s)
./status.sh ztktivcv7u
Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u
-----
qdisc fq_codel 0: root refcnt 2 limit 10240p flows 1024 quantum 2814 target 5ms interval 100ms memory_limit 32Mb ecn drop_batch 64
-----
```

enlace_limitado.sh

Simula un enlace de bajo ancho de banda con latencia moderada (512 kbit/s + 50 ms).

```
# Activar
sudo ./netem/enlace_limitado.sh <ZT_IFACE>
```

```
# Comando equivalente:
sudo tc qdisc add dev <ZT_IFACE> root netem rate 512kbit delay 50ms
```

```
# Desactivar
sudo ./netem/baseline.sh <ZT_IFACE>
```

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.059s)
./baseline.sh ztktivcv7u
Limpiando reglas de red en la interfaz: ztktivcv7u
Red restaurada a su estado base (sin degradación).

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.053s)
./enlace_limitado.sh ztktivcv7u
Configurando enlace limitado en la interfaz: ztktivcv7u
Escenario activado: Ancho de banda 512kbit + Delay 50ms.

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.043s)
./status.sh ztktivcv7u
Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u
-----
qdisc netem 8003: root refcnt 2 limit 1000 delay 50ms rate 512Kbit
-----
```

chino@chino ssh ~

iperf3 -s

Server listening on 5201 (test #1)

Accepted connection from 10.223.113.47, port 46956

[5] local 10.223.113.235 port 5201 connected to 10.223.113.47 port 33970

[ID]	Interval		Transfer	Bitrate
[5]	0.00-1.00	sec	0.00 Bytes	0.00 bits/sec
[5]	1.00-2.00	sec	0.00 Bytes	0.00 bits/sec
[5]	2.00-3.00	sec	128 KBytes	1.05 Mbits/sec
[5]	3.00-4.00	sec	0.00 Bytes	0.00 bits/sec
[5]	4.00-5.00	sec	128 KBytes	1.05 Mbits/sec
[5]	5.00-6.00	sec	0.00 Bytes	0.00 bits/sec
[5]	6.00-7.00	sec	128 KBytes	1.05 Mbits/sec
[5]	7.00-8.00	sec	0.00 Bytes	0.00 bits/sec
[5]	8.00-9.00	sec	128 KBytes	1.05 Mbits/sec
[5]	9.00-10.00	sec	0.00 Bytes	0.00 bits/sec
[5]	10.00-11.00	sec	128 KBytes	1.05 Mbits/sec
[5]	11.00-12.00	sec	0.00 Bytes	0.00 bits/sec
[5]	12.00-13.00	sec	128 KBytes	1.05 Mbits/sec
[5]	13.00-14.00	sec	0.00 Bytes	0.00 bits/sec
[5]	14.00-14.34	sec	0.00 Bytes	0.00 bits/sec

[ID]	Interval		Transfer	Bitrate	
[5]	0.00-14.34	sec	768 KBytes	439 Kbits/sec	receiver

Server listening on 5201 (test #2)

🔍 🔍 🔍

🔍 Search sessions, agents, files...

> ssh

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insert

iperf3 -c 10.223.113.235

Connecting to host 10.223.113.235, port 5201

[5] local 10.223.113.47 port 33970 connected to 10.223.113.235 port 5201

[ID]	Interval		Transfer	Bitrate	Retr	Cwnd
[5]	0.00-1.00	sec	128 KBytes	1.05 Mbits/sec	0	75.1 KBytes
[5]	1.00-2.00	sec	128 KBytes	1.05 Mbits/sec	0	77.8 KBytes
[5]	2.00-3.00	sec	0.00 Bytes	0.00 bits/sec	0	80.5 KBytes
[5]	3.00-4.00	sec	0.00 Bytes	0.00 bits/sec	0	88.6 KBytes
[5]	4.00-5.00	sec	256 KBytes	2.10 Mbits/sec	0	110 KBytes
[5]	5.00-6.00	sec	128 KBytes	1.05 Mbits/sec	0	140 KBytes
[5]	6.00-7.00	sec	128 KBytes	1.05 Mbits/sec	0	169 KBytes
[5]	7.00-8.00	sec	0.00 Bytes	0.00 bits/sec	0	199 KBytes
[5]	8.00-9.00	sec	256 KBytes	2.10 Mbits/sec	0	231 KBytes
[5]	9.00-10.00	sec	0.00 Bytes	0.00 bits/sec	0	263 KBytes

[ID]	Interval		Transfer	Bitrate	Retr	
[5]	0.00-10.00	sec	1.00 MBytes	839 Kbits/sec	0	sender
[5]	0.00-14.34	sec	768 KBytes	439 Kbits/sec		receiver

iperf Done.

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insert

./baseline.sh ztktivcv7u

Limpiando reglas de red en la interfaz: ztktivcv7u

Red restaurada a su estado base (sin degradación).

falla_nodo.sh

Simula la caída de un nodo edge deteniendo su contenedor. No usa tc, opera sobre Docker.

Activar (detener el contenedor)

./netem/falla_nodo.sh <NOMBRE_CONTENEDOR>

Comando equivalente:

docker stop <NOMBRE_CONTENEDOR>

Desactivar (reactivar el contenedor)

docker start <NOMBRE_CONTENEDOR>

```
chino@chino ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 1 file changed, 6 deletions(-) (20.53s)
./falla_nodo.sh chino-worker
Simulando falla crítica en el nodo: chino-worker (Pod: edge-daemonset-b7nrj)...
Desconectando el nodo lógicamente por 20 segundos...
node/chino-worker unlabeled
Forzando terminación del pod...
Warning: Immediate deletion does not wait for confirmation that the running resource has been terminated. The resource may continue to run on the cluster indefinitely.
pod "edge-daemonset-b7nrj" force deleted from default namespace
.....

chino@chino  ssh  ~/Pipeline_Distribuido/netem  feature/k3s  1 • -6

chino@chino ssh ~
sudo kubectl logs -f -l app=coordinador
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 31.53 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
2026-05-10T03:45:25.479231Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino-worker
| Promedio Muestras: 23.70 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
2026-05-10T03:45:27.267895Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 28.27 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
2026-05-10T03:45:35.490323Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino-worker
| Promedio Muestras: 25.79 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
2026-05-10T03:45:37.278506Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 27.94 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
2026-05-10T03:45:46.701149Z ERROR coordinador:
[!] ALERTA: Nodo Edge 'chino-worker' CAÍDO (No hay heartbeats en >10s)
2026-05-10T03:45:47.288905Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 31.55 °C
| Número de muestras: 10
| Anomalia Detectada: false
=====
```

```

2026-05-10T03:45:46.701149Z ERROR coordinador:
[!] ALERTA: Nodo Edge 'chino-worker' CAÍDO (No hay heartbeats en >10s)
2026-05-10T03:45:47.288905Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 31.55 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:45:57.298569Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 27.82 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:07.308744Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 26.78 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:17.318589Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 27.43 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:27.327028Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 25.01 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:37.338872Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 24.25 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:47.349129Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 27.49 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T03:46:51.053070Z INFO coordinador:
[+] INFO: Nodo Edge 'chino-worker' CONECTADO / RECONECTADO
2026-05-10T03:46:57.359661Z INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: chino
| Promedio Muestras: 26.92 °C
| Número de muestras: 10
| Anomalía Detectada: false

```

latencia_iot.sh

Simula la latencia típica de un enlace IoT (delay 80 ms con jitter de 20 ms, distribución normal).

Activar

sudo ./netem/latencia_iot.sh <ZT_IFACE>

Comando equivalente:

sudo tc qdisc add dev <ZT_IFACE> root netem delay 80ms 20ms distribution normal

Desactivar

sudo ./netem/baseline.sh <ZT_IFACE>

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.051s)
./latencia_iot.sh ztktivcv7u

Configurando latencia IoT en la interfaz: ztktivcv7u
Escenario activado: Delay 80ms + Jitter 20ms.

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.048s)
./status.sh ztktivcv7u

Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u
-----
qdisc netem 8001: root refcnt 2 limit 1000 delay 80ms 20ms
-----
```

```
Accepted connection from 10.223.113.47, port 57916
[ 5] local 10.223.113.235 port 5201 connected to 10.223.113.47 port 57918
[ ID] Interval            Transfer            Bitrate
[ 5] 0.00-1.00    sec      384 KBytes      3.14 Mbits/sec
[ 5] 1.00-2.00    sec      640 KBytes      5.25 Mbits/sec
[ 5] 2.00-3.00    sec      896 KBytes      7.34 Mbits/sec
[ 5] 3.00-4.00    sec      1.12 MBytes     9.42 Mbits/sec
[ 5] 4.00-5.00    sec      1.38 MBytes     11.5 Mbits/sec
[ 5] 5.00-6.00    sec      2.12 MBytes     17.8 Mbits/sec
[ 5] 6.00-7.00    sec      3.38 MBytes     28.3 Mbits/sec
[ 5] 7.00-8.00    sec      5.12 MBytes     43.0 Mbits/sec
[ 5] 8.00-9.00    sec      7.38 MBytes     61.9 Mbits/sec
[ 5] 9.00-10.00   sec      9.88 MBytes     82.8 Mbits/sec
[ 5] 10.00-10.08  sec      896 KBytes      90.4 Mbits/sec
-----
[ ID] Interval            Transfer            Bitrate
[ 5] 0.00-10.08   sec      33.1 MBytes     27.6 Mbits/sec
-----
Server listening on 5201 (test #3)
```

```
Server listening on 5201 (test #3)
```



```
> ssh tencia_iot.sh ztktivcv7u
```

Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u

```
qdisc netem 8001: root refcnt 2 limit 1000 delay 80ms 20ms
```

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insert
iperf3 -c 10.223.113.235
```

[5]	local 10.223.113.47 port 57918 connected to 10.223.113.235 port 5201					
[ID]	Interval		Transfer	Bitrate	Retr	Cwnd
[5]	0.00-1.00	sec	512 KBytes	4.19 Mb/s	2	48.3 KBytes
[5]	1.00-2.00	sec	768 KBytes	6.30 Mb/s	0	64.4 KBytes
[5]	2.00-3.00	sec	896 KBytes	7.34 Mb/s	0	83.2 KBytes
[5]	3.00-4.00	sec	1.12 MBytes	9.44 Mb/s	0	99.3 KBytes
[5]	4.00-5.00	sec	1.38 MBytes	11.5 Mb/s	0	148 KBytes
[5]	5.00-6.00	sec	2.38 MBytes	19.9 Mb/s	0	233 KBytes
[5]	6.00-7.00	sec	3.75 MBytes	31.5 Mb/s	0	357 KBytes
[5]	7.00-8.00	sec	5.38 MBytes	45.1 Mb/s	0	523 KBytes
[5]	8.00-9.00	sec	8.25 MBytes	69.2 Mb/s	0	741 KBytes
[5]	9.00-10.00	sec	10.1 MBytes	84.8 Mb/s	0	1012 KBytes
[ID]	Interval		Transfer	Bitrate	Retr	
[5]	0.00-10.00	sec	34.5 MBytes	28.9 Mb/s	2	sender
[5]	0.00-10.08	sec	33.1 MBytes	27.6 Mb/s		receiver

```
iperf Done.
```

perdida_paquetes.sh

Aplica una pérdida de paquetes del 8% sobre la interfaz ZT.

Activar

sudo ./netem/perdida_paquetes.sh <ZT_IFACE>

Comando equivalente:

sudo tc qdisc add dev <ZT_IFACE> root netem loss 8%

Desactivar

sudo ./netem/baseline.sh <ZT_IFACE>

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.046s)
./perdida_paquetes.sh ztktivcv7u

Configurando pérdida de paquetes en la interfaz: ztktivcv7u
Escenario activado: Pérdida de paquetes 8%.

chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.047s)
./status.sh ztktivcv7u
Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u
-----
qdisc netem 8002: root refcnt 2 limit 1000 loss 8%
-----
```

status.sh

Muestra el estado actual de las reglas tc qdisc. No activa ni desactiva nada; es solo de inspección.

Consultar estado

./netem/status.sh

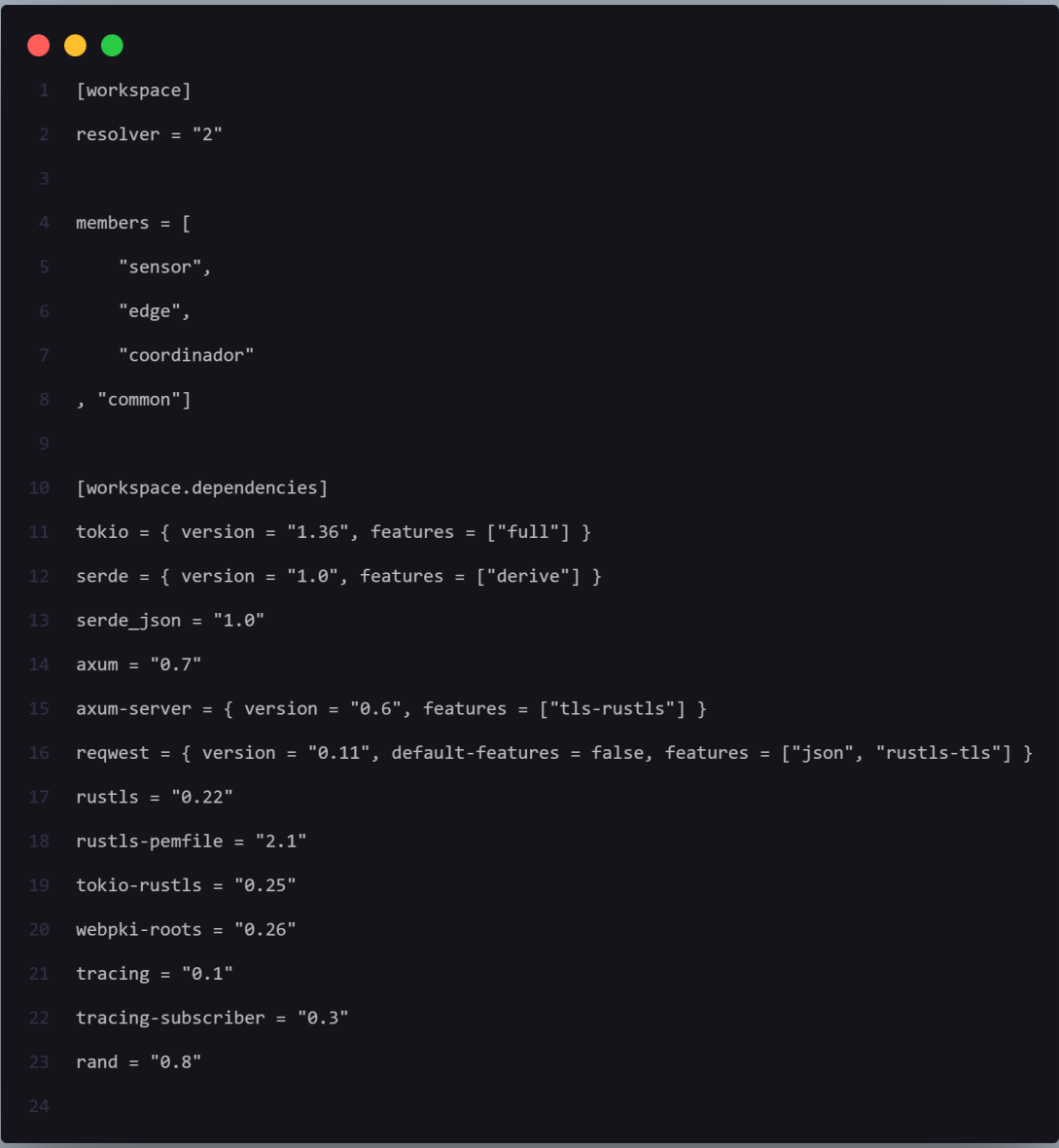
Comando equivalente (por interfaz específica):

sudo tc qdisc show dev <ZT_IFACE>

```
chino@chino-worker ssh ~/Pipeline_Distribuido/netem git:(feature/k3s) 7 files changed, 86 insertions(+), 24 deletions(-) (0.047s)
./status.sh ztktivcv7u
Estado actual de las reglas de red (tc qdisc) en la interfaz: ztktivcv7u
-----
qdisc netem 8001: root refcnt 2 limit 1000 delay 80ms 20ms
-----
```

Implementación base en Rust

Cargo.toml



```
1  [workspace]
2  resolver = "2"
3
4  members = [
5      "sensor",
6      "edge",
7      "coordinador"
8  , "common"]
9
10 [workspace.dependencies]
11 tokio = { version = "1.36", features = ["full"] }
12 serde = { version = "1.0", features = ["derive"] }
13 serde_json = "1.0"
14 axum = "0.7"
15 axum-server = { version = "0.6", features = ["tls-rustls"] }
16 reqwest = { version = "0.11", default-features = false, features = ["json", "rustls-tls"] }
17 rustls = "0.22"
18 rustls-pemfile = "2.1"
19 tokio-rustls = "0.25"
20 webpki-roots = "0.26"
21 tracing = "0.1"
22 tracing-subscriber = "0.3"
23 rand = "0.8"
24
```

Estructuras de mensaje con serde

```
1 use serde::{Deserialize, Serialize};
2
3 #[derive(Serialize, Deserialize, Debug, Clone)]
4 pub struct SensorReading {
5     pub sensor_id: String,
6     pub timestamp_ms: u64,
7     pub value: f64,
8     pub unit: String,
9 }
10
11 #[derive(Serialize, Deserialize, Debug, Clone)]
12 pub struct EdgeReport {
13     pub edge_id: String,
14     pub window_avg: f64,
15     pub anomaly_detected: bool,
16     pub sample_count: u32,
17     pub latency_ms: u64,
18 }
19
20 #[derive(Serialize, Deserialize, Debug, Clone)]
21 pub struct CoordStatus {
22     pub active_edges: u32,
23     pub total_readings: u64,
24     pub anomalies_last_min: u32,
25     pub uptime_s: u64,
26 }
27
28 #[derive(Serialize, Deserialize, Debug, Clone)]
29 pub struct Heartbeat {
30     pub node_id: String,
31     pub role: String,
32     pub timestamp_ms: u64,
33 }
34
```

Prototipo

```
2026-05-10T02:42:40.675596Z  INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: edge-1
| Promedio Muestras: 25.32 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T02:42:50.681808Z  INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: edge-1
| Promedio Muestras: 28.28 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T02:43:00.690104Z  INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: edge-1
| Promedio Muestras: 29.83 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T02:43:10.699731Z  INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: edge-1
| Promedio Muestras: 26.04 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
2026-05-10T02:43:20.709330Z  INFO coordinador:
===== [ COORDINADOR ] =====
| Reporte recibido de: edge-1
| Promedio Muestras: 25.32 °C
| Número de muestras: 10
| Anomalía Detectada: false
=====
```

```
2026-05-10T02:44:30.768844Z  INFO edge:
----- [ EDGE: Buffer ] -----
| Sensor: S2
| Valor: 33.48 °C
| Guardado en buffer (9/10 muestras)
-----
2026-05-10T02:44:30.773031Z  INFO edge:
===== [ EDGE: REPORTE LISTO ] =====
| Promedio de 10 muestras calculado: 30.19 °C
| Enviando reporte consolidado al Coordinador...
=====
2026-05-10T02:44:32.769798Z  INFO edge:
----- [ EDGE: Buffer ] -----
| Sensor: S2
| Valor: 27.75 °C
| Guardado en buffer (1/10 muestras)
-----
2026-05-10T02:44:32.775243Z  INFO edge:
----- [ EDGE: Buffer ] -----
| Sensor: S1
| Valor: 39.03 °C
| Guardado en buffer (2/10 muestras)
-----
2026-05-10T02:44:34.771936Z  INFO edge:
----- [ EDGE: Buffer ] -----
| Sensor: S2
| Valor: 33.47 °C
| Guardado en buffer (3/10 muestras)
-----
2026-05-10T02:44:34.777016Z  INFO edge:
----- [ EDGE: Buffer ] -----
| Sensor: S1
| Valor: 23.91 °C
| Guardado en buffer (4/10 muestras)
-----
```

```
2026-05-10T02:44:42.784036Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 19.65 °C
=====
2026-05-10T02:44:44.785831Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 20.51 °C
=====
2026-05-10T02:44:46.788101Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 34.29 °C
=====
2026-05-10T02:44:48.790355Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 24.25 °C
=====
2026-05-10T02:44:50.791168Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 25.33 °C
=====
2026-05-10T02:44:52.793481Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 32.97 °C
=====
2026-05-10T02:44:54.796339Z  INFO sensor:
===== [ SENSOR: S1 ] =====
| Enviando lectura de temperatura...
| Valor: 28.37 °C
=====
```

```
2026-05-10T02:45:06.798778Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 33.47 °C
=====
2026-05-10T02:45:08.801045Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 35.80 °C
=====
2026-05-10T02:45:10.802300Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 38.45 °C
=====
2026-05-10T02:45:12.803943Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 15.57 °C
=====
2026-05-10T02:45:14.805341Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 29.91 °C
=====
2026-05-10T02:45:16.809112Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 16.82 °C
=====
2026-05-10T02:45:18.810705Z  INFO sensor:
===== [ SENSOR: S2 ] =====
| Enviando lectura de temperatura...
| Valor: 33.82 °C
=====
```

Heartbeat

```
xansaul@spoke1 ssh ~/Pipeline-Distribuido git:(feature/k3s)
curl -i -X POST http://10.42.0.12:3001/api/heartbeat
HTTP/1.1 200 OK
content-length: 0
date: Sun, 10 May 2026 02:52:30 GMT
```

Plan de ejecución

- Implementar el sistema completo de tolerancia a fallos, incluyendo detección de desconexión de nodos edge mediante heartbeats y reconexión automática con reintentos.
- Completar la recolección y visualización de métricas de rendimiento en el coordinador, así como la detección de anomalías.
- Implementación de comunicación segura mediante certificados mTLS entre los nodos del sistema, lo que puede generar complejidad adicional en la configuración, validación y renovación de certificados dentro de los contenedores y servicios distribuidos.
- Finalizar los scripts de automatización para los escenarios de red degradada utilizando tc netem, incluyendo pérdida de paquetes, limitación de ancho de banda y simulación de caída de nodos.
- Realizar pruebas integrales del pipeline distribuido sobre la red ZeroTier para validar el comportamiento del sistema bajo distintas condiciones de red.

Pruebas de tolerancia a fallos

- Se simulará la caída de un nodo edge deteniendo manualmente su contenedor dentro de Kubernetes para verificar que el coordinador detecte la ausencia de heartbeats en menos de 10 segundos.
- Se probará la reconexión automática reiniciando el contenedor edge y verificando que se retome el envío de datos sin intervención manual.

- Se aplicarán escenarios de pérdida de paquetes y latencia utilizando tc netem para evaluar el comportamiento del sistema ante degradación de red.
- Se validará el manejo de arranque desordenado iniciando sensores y nodos edge antes del coordinator para comprobar que los mecanismos de reintento funcionen correctamente.
- Se utilizará iperf3 para medir el rendimiento base de la red y comparar el impacto de cada escenario de degradación.

Medición de métricas de rendimiento

- Se registrará el throughput del sistema midiendo la cantidad de mensajes procesados por segundo en el coordinator y por cada nodo edge.
- La latencia extremo a extremo se calculará comparando el timestamp generado por el sensor con el momento de recepción del mensaje en el coordinator.
- Se utilizará iperf3 antes y después de aplicar escenarios de tc netem para comparar el impacto de la degradación en el rendimiento de la red.
- Se recopilarán logs del sistema y métricas exportadas por el coordinator para generar tablas y gráficas comparativas entre los distintos escenarios evaluados.
- Se analizará la tasa de anomalías detectadas y la cantidad estimada de mensajes perdidos durante pruebas con pérdida de paquetes y limitación de ancho de banda.

Riesgos técnicos identificados

- Riesgo de race conditions durante las reconexiones automáticas de los nodos edge debido al manejo concurrente de tareas asíncronas en Rust.
- Desincronización de relojes entre nodos distribuidos, lo que puede afectar el cálculo preciso de la latencia extremo a extremo.
- Acumulación excesiva de mensajes en buffers cuando la red presenta alta latencia o pérdida de paquetes.
- Problemas relacionados con el orden de arranque de los servicios distribuidos, especialmente cuando sensores o edge nodes inician antes que el coordinador.

Metodología Ágil

Para el desarrollo del sistema adoptamos un **Scrum ligero**, ajustado a las necesidades del equipo y al tiempo disponible. Priorizamos la entrega funcional sobre la documentación extensa. Las reuniones se mantuvieron enfocadas en el avance, identificando bloqueos y asignando tareas de manera dinámica.

Roles

Casillas Rendón Jennyfher Jacqueline: Desarrollador, Responsable de la documentación.

Huerta Murillo Alejandro Saul: Desarrollador.

Lona Avalos Cristian Josué: Desarrollador, Responsable técnico (RT).

Cronograma

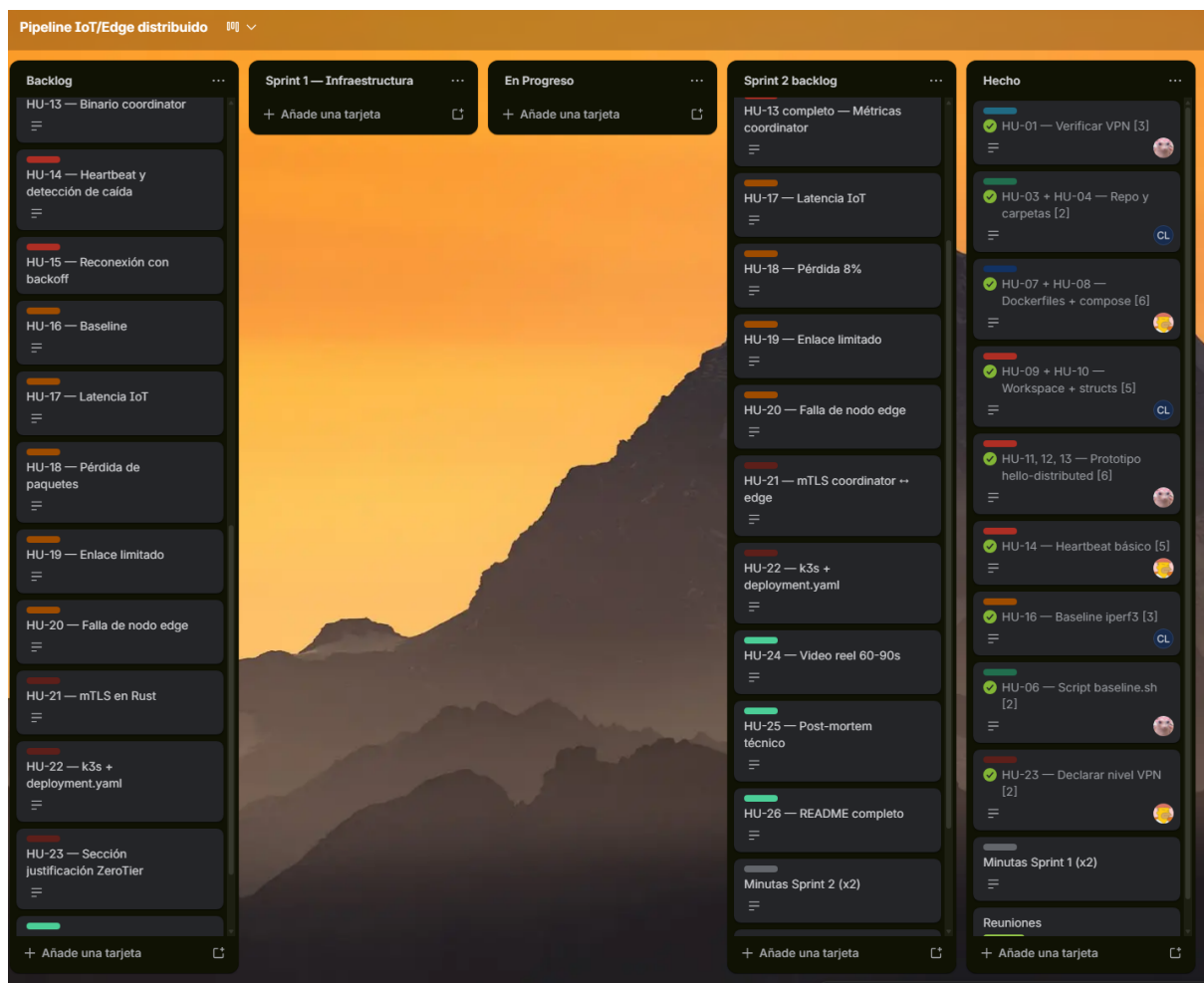
Sprint 1	Metas
tc netem baseline y prototipo Rust (hello distributed)	14 abril - 06 de mayo
Entrega de avance	07 de mayo

Product Backlog (Historias de Usuario)

Prioridad	Historia de Usuario	Rol	Objetivo	Esfuerzo relativo
1	Pipeline básico funcionando	Desarrollador	Implementar flujo completo sensor → edge → coordinator	(alto, base del sistema)
2	Detección de fallas en nodos	Coordinator	Detectar desconexión de un nodo en <10s con heartbeats	(medio)

3	Reconexión automática de edge nodes	Nodo Edge	Reconectarse al coordinator al restablecer conexión	(medio)
4	Simulación de red degradada	Responsable técnico	Aplicar escenarios con tc netem (latencia, pérdida, ancho de banda)	(alto, requiere configuración avanzada)

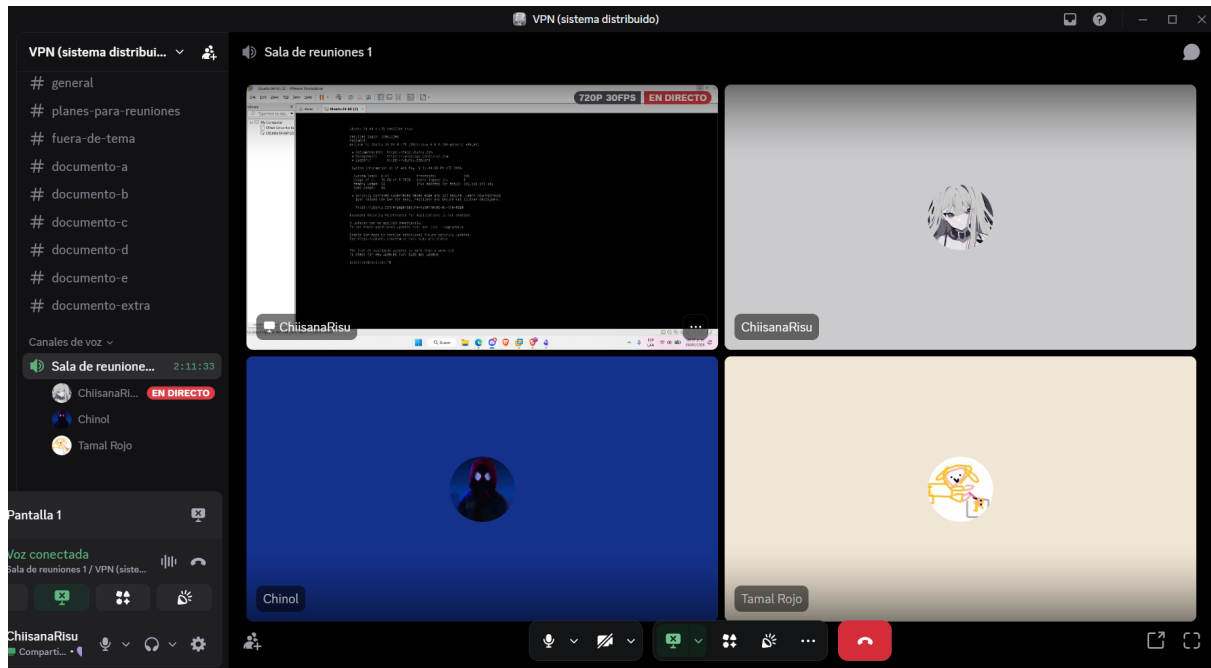
Sprint Backlog



Minutas de Reunión

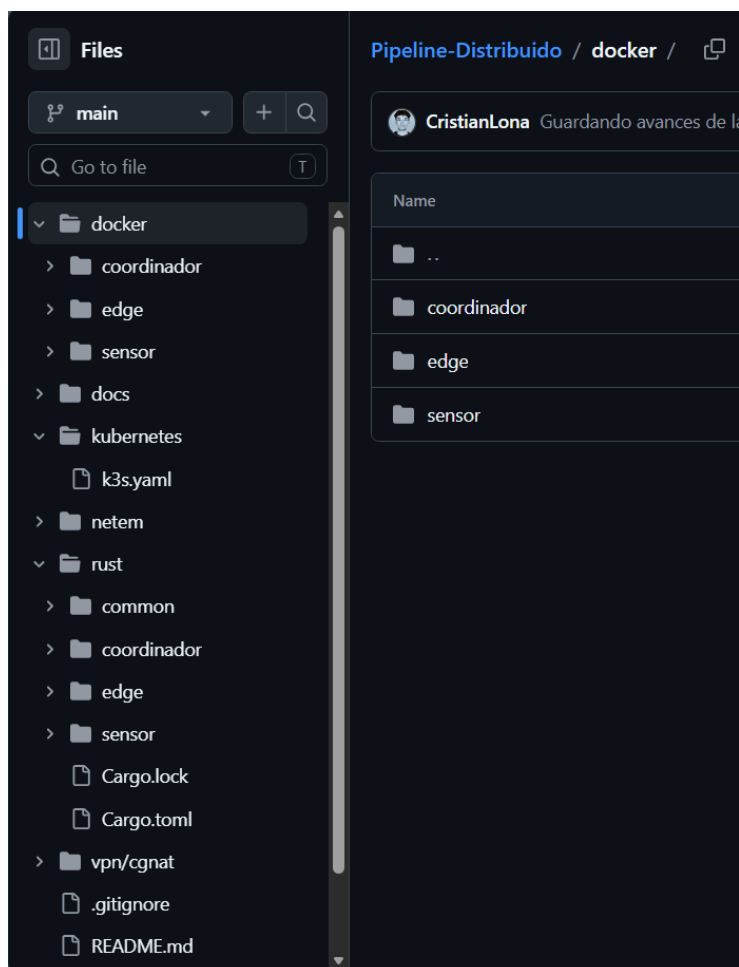
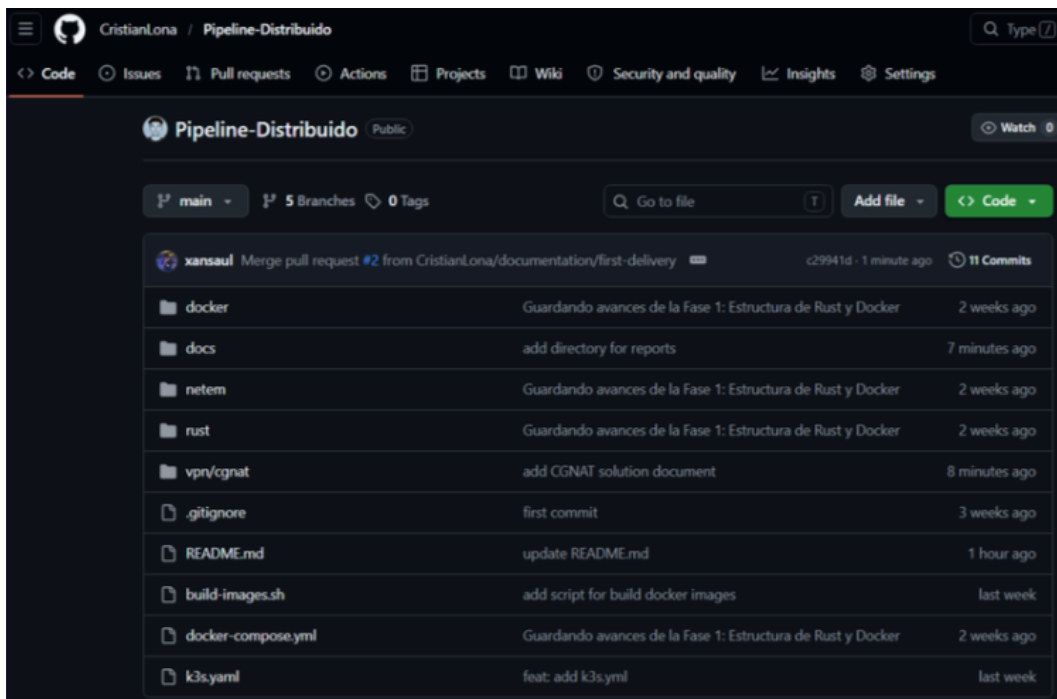
- Número de minuta: 1
- Sprint: 1
- Fecha: 29/04/2026
- Hora de inicio: 19:02
- Hora de fin: 22:57
- Integrantes presentes: Todos
- Temas discutidos:
 - Conexión entre las distintas máquinas (pruebas)
 - La conexión se propuso usar zerotier ya que no pudimos conseguir una red local autoadministrada por temas del CGNAT, y la comunicación entre pods está establecida por la red interna que monta k3s.
- Decisiones tomadas:
 - Decidimos montar un proyecto de rust aprovechando el tema de poder compilar varios binarios, a su vez nos basamos estrictamente en las funciones que declaran los documentos de requisitos, en base a eso montamos los 3 tipos de contenedor, diseñamos la comunicación entre ellos con mensajes http tipo rest y ejecutamos.
 - Que nuestro compañero Cristian sea el nodo master.
- Tareas generadas:
 - Montar el cluster con los binarios de rust, ver que sirvan en el mismo nodo master. (responsable Cristian, fecha límite: 30/04/2026)
 - Si funciona en el nodo master -> montar con otros nodos como "agentes" los cuales se pueden controlar desde el nodo master. (responsable Saul, Jennyfher, fecha límite: 05/05/2026)
- Compromisos para la siguiente reunión:
 - El tener un checklist de todo lo que logramos implementar vs lo esperado, ver que hace falta e idear un plan para implementar de la manera más eficiente y ágil cada parte faltante.
- Evidencia:

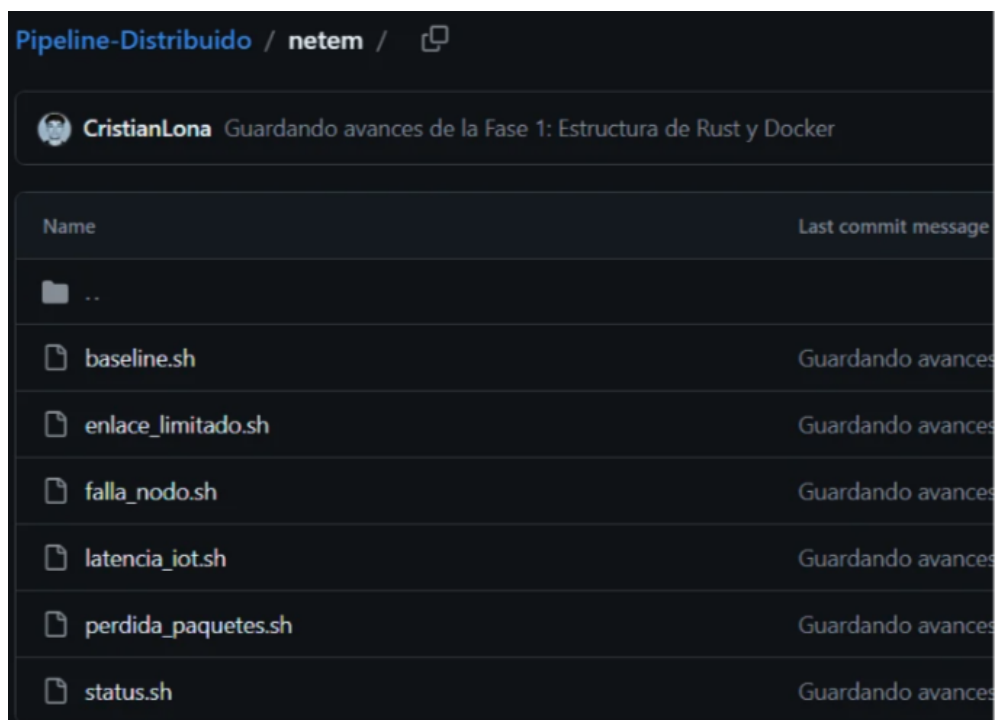
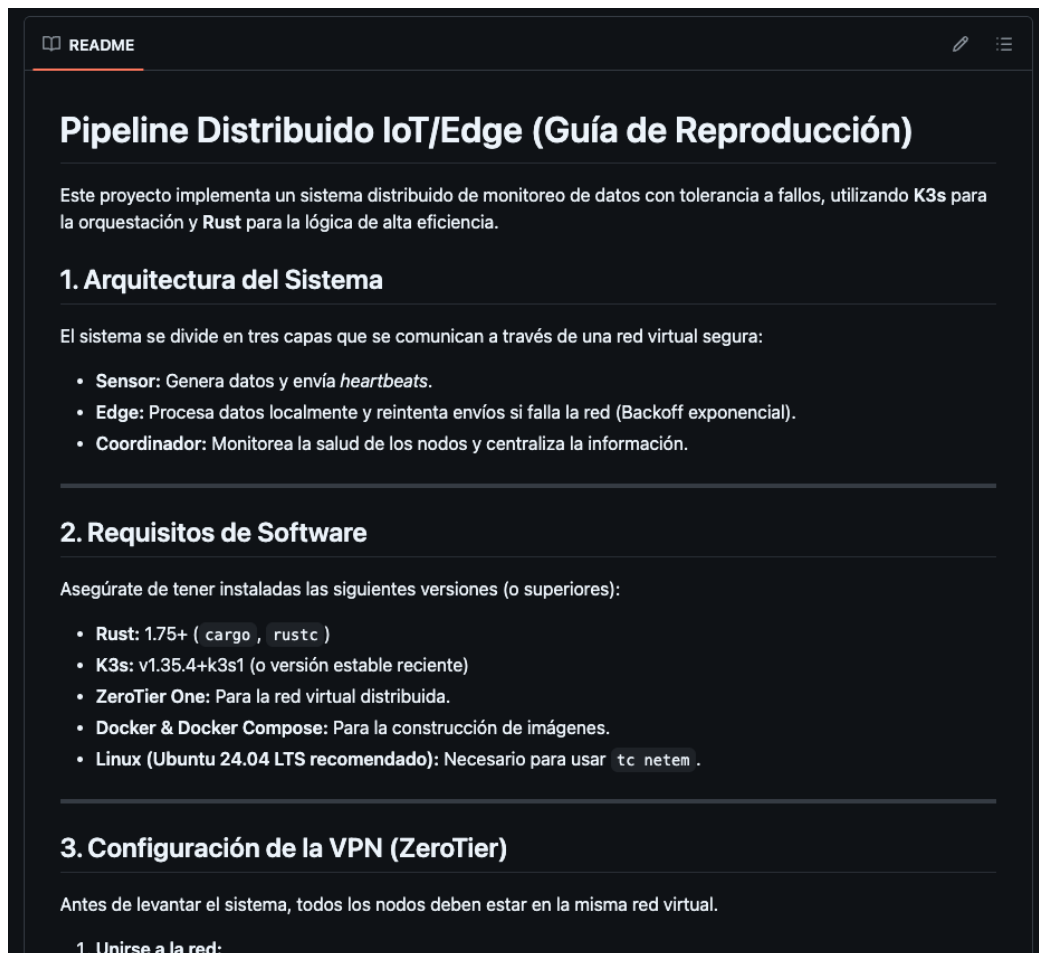
- Hacer pruebas del funcionamiento correcto del cluster. (responsable Cristian, fecha límite: 10/05/2026)
- Probar, auditar y mejorar los scripts tc netem. (responsable Cristian, Jennyfher, fecha límite: 13/05/2026)
- Compromisos para la siguiente reunión: Revisar los requerimientos para el Sprint 2 para crear y dividir las tareas.
- Evidencia:



Repositorio

link: <https://github.com/CristianLona/Pipeline-Distribuido>





```
Code Blame 21 lines (17 loc) · 241 Bytes

1  # Rust build artifacts
2  target/
3
4  # Llaves y certificados
5  *.key
6  *.pem
7  *.p12
8  *.crt
9  *.csr
10
11 # WireGuard config real (solo versiones sanitizadas en /vpn/)
12 wg0.conf
13
14 # Variables de entorno con credenciales
15 .env
16 .env.*
17
18 # Sistema
19 .DS_Store
20 *.swp
21 *~
```

Retos encontrados hasta el momento y cómo se resolvieron

Uno de los desafíos encontrados fue la comunicación de los nodos en la red de ZeroTier, ya que en ocasiones fallaba la respuesta de ping de forma aleatoria. Para solucionarlo, tenemos que reiniciar la máquina virtual o reinstalar el agente de k3s con la configuración correspondiente.

Otro desafío involucró el script de tc netem, diseñado para probar la caída de un pod y su posterior recuperación por parte de k3s. El inconveniente es que este proceso ocurría tan rápido que no quedaba registrado en los logs. Al intentar implementar un tiempo de espera para forzar su registro, el comportamiento resultaba impredecible dependiendo del nodo. Este detalle aún no se ha resuelto y está contemplado para una futura etapa de desarrollo.

Conclusión

Durante el desarrollo de este avance se logró establecer la base funcional de la infraestructura distribuida requerida para el proyecto, integrando tecnologías como Rust, Docker, Kubernetes y ZeroTier dentro de un entorno orientado a la simulación de un pipeline IoT/Edge tolerante a fallos. Uno de los principales logros alcanzados fue la implementación inicial del flujo completo entre sensores, nodos edge y coordinador, permitiendo demostrar el envío y procesamiento básico de datos de extremo a extremo.

Otro aspecto importante fue la integración de Kubernetes como plataforma de orquestación, facilitando el despliegue y administración de los contenedores que conforman el sistema distribuido. De igual manera, se realizaron pruebas iniciales con tc netem para simular condiciones de red degradadas, observando cómo factores como la latencia y la pérdida de paquetes pueden afectar el comportamiento del sistema.

Consideramos que logramos cubrir satisfactoriamente los requerimientos establecidos para el primer avance del proyecto, alcanzando los objetivos principales planteados para esta etapa y estableciendo una base sólida para continuar con el desarrollo del sistema.

Referencias

Docker Inc. (2024). Docker (Versión 24.0) [Software de computación].

<https://www.docker.com>

Canonical Ltd. (2024). Ubuntu Server (Versión 22.04 LTS) [Sistema operativo].

<https://ubuntu.com/download/server>

Microsoft Learn (s. f.). Virtual Private Networking: An Overview. Recuperado de

[https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-2000-server/bb742566\(v=technet.10\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-2000-server/bb742566(v=technet.10))

AWS. (s. f.). ¿Qué es Docker? Recuperado de <https://aws.amazon.com/es/docker/>

Docker. (s. f.). What is a Container? Recuperado de

<https://www.docker.com/resources/what-container/>